

CS201 Final

2 June 2024

Olcay Taner YILDIZ

Abstract— Write only **ONE** function per question. Do not check the input, assume that the input is correct. Unless noted, (i) the input data structure is not empty, (ii) time complexity of the algorithm does not matter, (iii) you can not use extra data structures including arrays, (iv) you can use any function given in the data structure, (v) you can not modify the data structure contents.

I. QUESTION (ALGORITHM ANALYSIS) (20 PTS)

```

1 int algorithm1(int N){
2   if (N == 0)
3     return 0;
4   int sum = 0;
5   for (int i = 0; i < N; i++)
6     for (int j = 0; j < N; j++)
7       for (int k = 0; k < N; k++)
8         sum++;
9   for (int i = 0; i < 2; i++)
10    for (int j = 0; j < 2; j++)
11      for (int k = 0; k < 2; k++)
12        sum += algorithm1(N / 2);
13   return sum;
14 }
```

What is the time complexity of algorithm1? Solve the recurrence equation without the master theorem.

II. QUESTION (LINKED LIST) (20 PTS)

Write the method in **LinkedList** class

```
void remove(const LinkedList& list2)
```

which removes the nodes that appear in the list2 from the original list. You are not allowed to use any methods from the LinkedList class. You can assume both the original list and list2 do not contain duplicate elements. Do not modify list2. Your method should run in $\mathcal{O}(N^2)$ time.

III. QUESTION (TREE) (12 PTS)

Write a recursive method in **TreeNode** class

```
void accumulate(int* a, int& index)
```

that accumulates all contents (integers as data field) in the tree in array a, where the values in the array are will be sorted. Use and modify index to store the integers

into correct positions. Your method should run in $\mathcal{O}(N)$ time. The function can be called like this:

```
int a[100];
int index = 0;
t.getRoot()->accumulateData(a, index);
```

IV. QUESTION (GRAPH) (20 PTS)

Write the method in linked list implementation of **Graph** class

```
Graph intersection(const Graph& g2, int v)
```

that returns a new graph formed by adding edges which exist both in the original graph and g2. You may assume both graphs are unweighted.

V. QUESTION (SORTING) (14 PTS)

Modify the original bubble sort such that

```
void sort2(int* A, int* B, int* C, int N)
```

uses B as a secondary and C as a ternary key. If two elements in A are equal, algorithm uses B as a secondary key. If also their keys in B are equal, the algorithm will then resort to C array. Do not modify arrays B and C. N is the number of elements in the array. Your modified method should run in $\mathcal{O}(N^2)$ time.

VI. QUESTION (SORTING) (14 PTS)

Implement a modified version of partition algorithm to write a method in **QuickSort** class

```
void oddsBeforeEvens(int* A, int N)
```

which moves the odd numbers before even ones in A in one pass. The algorithm is as follows:

1. At the beginning of the algorithm, let say we have two indexes i and j, showing the first and last elements respectively.
2. While i is less than j,
 - Increment i until you find an even number.
 - Decrement j until you find an odd number.
 - If $i < j$, swap the contents of i and j.
3. Continue with step 2.